Relational Algebra and Calculus

Ioana Manolescu
Inria and Ecole polytechnique



Review

- Conceptual Design: ER model
- Logical Design: ER to relational model
- Querying and manipulating data
  - Formal languages: relational algebra and calculus (today)
  - Practical language: SQL (starting next week)

Relational Query Languages

- Relational model allows simple, powerful *querying* of data.
- Relational query languages:
  - Declarative: say "what you want", not "how to get it"
  - Formal mathematical model
  - Query optimization

Formal Relational Query Languages

- Two mathematical languages form the basis for the "real" one, SQL, and for implementation:
  - *Relational Algebra*: operational, useful for representing execution plans
    - How the query is executed
  - *Relational Calculus*: declarative, useful for defining query semantics.
    - What the query means
- We will end by discussing the relationship between these two.

Relational Algebra



What is Relational Algebra

- Relational algebra:
  - Operands are relations.
  - Each operator takes as input 1 or 2 relations and produces a relation.
- Closure property: relational algebra is closed under the relational model.
  - Relational operators can be arbitrarily *composed*

Relational Algebra

- Basic operations:
  - *Selection* (σ) Selects a subset of rows from a relation.
  - *Projection* (π) Deletes unwanted columns from a relation.
  - *Cross-product* ($\times$) Allows us to combine two relations.
  - *Set-difference* ($-$, $\setminus$) Tuples in relation 1, but not in relation 2.
  - *Union* ($\cup$) Tuples in relation 1 and in relation 2.
- Additional operations:
  - *Join* ($\bowtie$), *Intersection* ($\cap$), *Division* (/), *Renaming* (ρ)
  - Can be derived from basic operators. Not essential, but useful

Example Instances

Sailors

S1

sid	sname	rating	age
22	dustin	7	45.0
31	lubber	8	55.5
58	rusty	10	35.0

S2

sid	sname	rating	age
28	yuppy	9	35.0
31	lubber	8	55.5
44	guppy	5	35.0
58	rusty	10	35.0

Reserves

R1

sid	bid	day
22	101	10/10/96
58	103	11/12/96

Projection

- Retain only attributes in the *projection list*; delete others.
- *Schema of result* contains exactly the fields in projection list.

S2

sid	sname	rating	age
28	yuppy	9	35.0
31	lubber	8	55.5
44	guppy	5	35.0
58	rusty	10	35.0

sname	rating
yuppy	9
lubber	8
guppy	5
rusty	10

$$\pi_{sname,rating}(S2)$$

Projection (contd.)

- Projection operator has to eliminate *duplicates*
  - Real systems typically don't do duplicate elimination *unless the user explicitly asks for it*.
    This is in order to avoid the cost of duplicate elimination.

S2

sid	sname	rating	age
28	yuppy	9	35.0
31	lubber	8	55.5
44	guppy	5	35.0
58	rusty	10	35.0

age
35.0
55.5

$\pi_{age}(S2)$

Selection

- Select rows that satisfy the *selection condition*; discard others.
- *Schema of result* identical to schema of input.

S2

sid	sname	rating	age
28	yuppy	9	35.0
31	lubber	8	55.5
44	guppy	5	35.0
58	rusty	10	35.0

sid	sname	rating	age
28	yuppy	9	35.0
58	rusty	10	35.0

$$\sigma_{rating>8}(S2)$$

Selection (continued)

- *Composition*: *result* relation of an operator can be the *input* to another operator.

sid	sname	rating	age
28	yuppy	9	35.0
58	rusty	10	35.0

$$\sigma_{rating>8}(S2)$$

sname	rating
yuppy	9
rusty	10

$$\pi_{sname,rating}(\sigma_{rating>8}(S2))$$

Union, Intersection, Set-Difference

- Set operations:
  - *Union* (∪)
  - *Intersection* (∩)
  - *Set difference* (- or \ )
- Take two input relations, which must have the same schema:
  - Same number of fields.
  - Corresponding fields have the the same type.
- The result has the same schema as the inputs

S1

sid	sname	rating	age
22	dustin	7	45.0
31	lubber	8	55.5
58	rusty	10	35.0

S2

sid	sname	rating	age
28	yuppy	9	35.0
31	lubber	8	55.5
44	guppy	5	35.0
58	rusty	10	35.0

Example Set Operations

S1

sid	sname	rating	age
22	dustin	7	45.0
31	lubber	8	55.5
58	rusty	10	35.0

S2

sid	sname	rating	age
28	yuppy	9	35.0
31	lubber	8	55.5
44	guppy	5	35.0
58	rusty	10	35.0

sid	sname	rating	age
22	dustin	7	45.0
31	lubber	8	55.5
58	rusty	10	35.0
44	guppy	5	35.0
28	yuppy	9	35.0

$S1 \cup S2$

Duplicate elimination: remove tuples that have same values in all attributes.

Example Set Operations

S1

sid	sname	rating	age
22	dustin	7	45.0
31	lubber	8	55.5
58	rusty	10	35.0

S2

sid	sname	rating	age
28	yuppy	9	35.0
31	lubber	8	55.5
44	guppy	5	35.0
58	rusty	10	35.0

sid	sname	rating	age
22	dustin	7	45.0

$S1 - S2$

sid	sname	rating	age
31	lubber	8	55.5
58	rusty	10	35.0

$S1 \cap S2$

Duplicates?

Cross-Product (aka Cartesian Product)

- S1 × R1: Each row of S1 is paired with each row of R1.

S1

sid	sname	rating	age
22	dustin	7	45.0
31	lubber	8	55.5
58	rusty	10	35.0

R1

sid	bid	day
22	101	10/10/96
58	103	11/12/96

S1 × R1

(sid)	sname	rating	age	(sid)	bid	day
22	dustin	7	45.0	22	101	10/10/96
22	dustin	7	45.0	58	103	11/12/96
31	lubber	8	55.5	22	101	10/10/96
31	lubber	8	55.5	58	103	11/12/96
58	rusty	10	35.0	22	101	10/10/96
58	rusty	10	35.0	58	103	11/12/96

Cross-Product (continued)

(sid)	sname	rating	age	(sid)	bid	day
22	dustin	7	45.0	22	101	10/10/96
22	dustin	7	45.0	58	103	11/12/96
31	lubber	8	55.5	22	101	10/10/96
31	lubber	8	55.5	58	103	11/12/96
58	rusty	10	35.0	22	101	10/10/96
58	rusty	10	35.0	58	103	11/12/96

- *Result schema* inherits all fields of S1 and R1.
  - *Conflict*: Both S1 and R1 have a field called *sid*.
- *Renaming operator*: $\rho\,(C(1 \to sid1, 5 \to sid2), S1 \times R1)$

Joins

- *Condition (theta) Join*: $R \bowtie_c S = \sigma_c (R \times S)$

(sid)	sname	rating	age	(sid)	bid	day
22	dustin	7	45.0	58	103	11/12/96
31	lubber	8	55.5	58	103	11/12/96

$$S1 \bowtie_{S1.sid < R1.sid} R1$$

- *Result schema* same as that of cross-product.
- But often fewer tuples, more efficient for computation.

Joins

- *Equi-Join*: A special case of condition join where the condition θ contains only *equalities*.

sid	sname	rating	age	bid	day
22	dustin	7	45.0	101	10/10/96
58	rusty	10	35.0	103	11/12/96

$$S1 \bowtie_{sid} R1$$

  - *Result schema* contains only *one copy* of fields for which equality is specified.

- *Natural Join* ($R \bowtie S$): equijoin on *all* common fields.

Division

- Not a basic operator, but useful for expressing queries:
  *Find sailors who have reserved **all** boats.*
- Let $A = \{x, y\}$; let $B = \{y\}$:
  - $A/B = \left\{ \langle x \rangle \mid \forall \langle y \rangle \in B, \exists \langle x, y \rangle \in A \right\}$
  - A/B contains *all* x tuples (sailors) such that for *every* y tuple (boat) in B, there is an xy tuple in A.
- In general, $A = \{x, y\}$, $B = \{y\}$, x and y can be any lists of fields, A/B = subset of {x}.

Examples of Division A/B

x *y*

sid	bid
s1	b1
s1	b2
s1	b3
s1	b4
s2	b1
s2	b2
s3	b2
s4	b2
s4	b4

A

y

bid
b2

B1

x

sid
s1
s2
s3
s4

A/B1

bid
b2
b4

B2

sid
s1
s4

A/B2

bid
b1
b2
b4

B3

sid
s1

A/B3

Why is it called division?

x *y*

sid	bid
s1	b1
s1	b2
s1	b3
s1	b4
s2	b1
s2	b2
s3	b2
s4	b2
s4	b4

A
"dividend"

"divisor"

bid
b2
b4

B2

sid
s1
s4

A/B2
"quotient"

Integer division:

$(A/B) \times B + r = A$

sid	bid
s1	b2
s1	b4
s4	b2
s4	b4

$(A/B2) \times B2 \cup$

"remainder"

sid	bid
s1	b3
s2	b1
s2	b2
s3	b4

$= A$

Example Schema

- **Sailors**(*sid*: integer, *sname*: string, *rating*: integer,
  age: integer)
- **Boats**(*bid*: integer, *color*: string)
- **Reserves**(*sid*: integer, *bid*: integer, *day*: date)

Find names of sailors having reserved boat #103

- **Sailors**(*sid*: integer, *sname*: string, *rating*: integer,
  age: integer)
- **Boats**(*bid*: integer, *color*: string)
- **Reserves**(*sid*: integer, *bid*: integer, *day*: date)

Find names of sailors having reserved boat #103

- Solution 1: $\pi_{sname}((\sigma_{bid=103} \text{Reserves}) \bowtie Sailors)$

- Solution 2: $\rho\ (Temp1,\ \sigma_{bid=103} \text{Reserves})$

  $\rho\ (Temp2,\ Temp1 \bowtie Sailors)$

  $\pi_{sname}(Temp2)$

- Solution 3: $\pi_{sname}(\sigma_{bid=103}(\text{Reserves} \bowtie Sailors))$

Equivalent algebraic expressions

Find names of sailors having reserved a red boat

- **Sailors**(*sid*: integer, *sname*: string, *rating*: integer,
  age: integer)
- **Boats**(*bid*: integer, *color*: string)
- **Reserves**(*sid*: integer, *bid*: integer, *day*: date)

- Boat color is only available in Boats
  - Thus, we need an extra join:

$$\pi_{sname}((\sigma_{color='red'} Boats) \bowtie Reserves \bowtie Sailors)$$

Find sailors having reserved a red or a green boat

- Identify all red or green boats, then find sailors who've reserved one of these boats:

$$\rho\ (Tempboats, (\sigma_{color='red' \vee color='green'} Boats))$$

$$\pi_{sname}(Tempboats \bowtie Reserves \bowtie Sailors)$$

- Alternative: define Tempboats using *union*. How?

Find sailors having reserved a red and a green boat

- A single selection will not work.
- Instead, *intersect* sailors having reserved red boats and those having reserved green boats.

sid is a *key* for Sailors

$$\rho\ (Tempred, \pi_{sid}((\sigma_{color='red'} Boats) \bowtie \text{Re}serves))$$

$$\rho\ (Tempgreen, \pi_{sid}((\sigma_{color='green'} Boats) \bowtie \text{Re}serves))$$

$$\pi_{sname}((Tempred \cap Tempgreen) \bowtie Sailors)$$

Find the names of sailors having reserved all boats

- Use division A/B; carefully choose schemas of input relations, A ={x,y}, B={y}:

$$\rho\,(Tempsids, (\pi_{sid,bid}\text{Re}serves) / (\pi_{bid} Boats))$$

$$\pi_{sname}(Tempsids \bowtie Sailors)$$

- To find sailors who've reserved all red boats:

..... $/\ \pi_{bid}(\sigma_{color='red'} Boats)$

Relational Calculus



Relational Calculus

- **Declarative** formalism for expressing queries
- Uses *variables, constants, comparison operators, logical connectives* and *quantifiers*.
- Two forms: *Tuple relational calculus* (TRC) and *Domain relational calculus* (DRC).
  - *TRC*: Variables range over (i.e., get bound to) *tuples*.
  - *DRC*: Variables range over *domain elements* (= field values).
  - Both are simple subsets of **first-order logic**.
- *Formulas:* expressions in the calculus.
- *An answer tuple*: an assignment of constants to variables that make the formula evaluate to *true*.

Relational Calculus

- *Query* has the form:

$$\{\langle x1, x2, \ldots, xn \rangle \mid p(\langle x1, x2, \ldots, xn \rangle)\}$$

domain variables, or *constants*

formula

- *Answer* includes all *tuples* $\langle x1, x2, \ldots, xn \rangle$ that make the formula $p(\langle x1, x2, \ldots, xn \rangle)$ *true*.

Tuple Relational Calculus

- *Query* has the form:

$$\{\langle x1, x2, \ldots, xn \rangle \mid p(\langle x1, x2, \ldots, xn \rangle)\}$$

- *Answer* includes all tuples $\langle x1, x2, \ldots, xn \rangle$ that make the *formula* $p(\langle x1, x2, \ldots, xn \rangle)$ be *true*.

- *Formula* is recursively defined, starting with simple *atomic formulas* (getting tuples from relations or making comparisons of values), and building larger formulas using the *logical connectives*.

TRC Formulas

- *Atomic formula:*
  - $X \in Rname$, or X.a *op* Y.b, or X.a *op* constant
  - *op* is one of <,>,=,≤,≥,≠
- *Formula:*
  - an atomic formula, or
  - $\neg p, p \wedge q, p \vee q$, where p and q are formulas, or
  - $\exists X(p(X))$, where variable X is *free* in p(X), or
  - $\forall X(p(X))$, where variable X is *free* in p(X)
- The use of quantifiers $\exists X$ and $\forall X$ is said to *bind* X.

Find sailors rated > 7 who have reserved a red boat

Relational Algebra:

$$\pi_{sname}((\sigma_{color='red'}Boats) \bowtie Reserves \bowtie (\sigma_{rating>7}Sailors))$$

Tuple Relational Calculus:

$$\{P \mid \exists S\,(S \in Sailors \wedge \exists B\,(B \in Boats \wedge \exists R\,(R \in Reserves \wedge S.\text{rating} > 7 \wedge \underline{S.\text{name}=P.\text{name}} \wedge B.\text{color}=\text{'red'} \wedge \underline{S.\text{sid}=R.\text{sid} \wedge R.\text{bid}=B.\text{bid}}))) \}$$

The joins are the underlined equalities

Find sailors rated > 7 who have reserved a red boat

Relational Algebra:

$$\pi_{sname}((\sigma_{color='red'}Boats) \bowtie Reserves \bowtie (\sigma_{rating>7}Sailors))$$

Tuple Relational Calculus (shorter, easy to read notation):

$$\{P \mid \exists S \in Sailors, B \in Boats, R \in Reserves$$
$$S.\text{rating} > 7 \wedge \underline{S.\text{name}=P.\text{name}} \wedge B.\text{color}=\text{'red'} \wedge$$
$$\underline{S.\text{sid}=R.\text{sid} \wedge R.\text{bid}=B.\text{bid}}\}$$

The joins are the underlined equalities

Find names of sailors rated > 7 who've reserved all boats

Tuple Relational Calculus:

$$\{P \mid \exists S \in Sailors\ (S.\text{rating} > 7 \wedge S.\text{name}=P.\text{name} \wedge \forall B \in Boats\ (\exists R \in Reserves\ (S.\text{sid}=R.\text{sid} \wedge R.\text{bid}=B.\text{bid}))))\}$$

Questions



Relational Algebra and Calculus:
how do their expressive powers compare?



Expressive power comparison

- Relational algebra translates into relational calculus.
  - Exercise: propose a proof for this
- What about the opposite direction?

Unsafe Relational Calculus Queries

- Some syntactically correct calculus queries can have an infinite number of answers! Such queries are called *unsafe*.
  - e.g., $\{S \mid \neg(S \in Sailors)\}$
  - *Users do not expect infinite answers from a finite database*

- Another "unpleasant" query: {X| ∀ Y, R(X,Y) }
  - If Y has an infinite domain and R is finite, this is always empty
  - *Users should not have to be aware of which domains are finite or not*

- **Intuition**: users only expect *answers from the database*
- Infinite answers are not practical even when correct

Active-domain interpretation of RC

- Interpretation of RC which ensures safety

- Finite database instance I, query q defined by formula φ, with the free variables *free(φ)*
- Active domain for a query q and instance I: *adom(q,I)*
  - **All the tuples made of *I* constants and *q* constants**
  - *adom(q, I)* is **finite!**
- Active domain interpretation: query variables (and in particular *free(φ)*) can only be tuples of values from *adom(q, I)*

Active-domain interpretation (cont'd)

- Recall: *adom(q,I)* = all tuples made from *I* and *q* constants
- Let *υ* be a **valuation** of *q*'s variables into *adom(q, I)*
  - *υ* assigns a tuple from *adom(q,I))* to each *q* variable
  - In general, zero, one or many valuations exist for one query
- Query *q* defined by formula *φ*, free vars. *free(q)*, returns *head(q)* (tuple whose attributes come from the body of *q*)

- *I* **satisfies** *φ* for the valuation *υ* under the active-domain interpretation, denoted $I \models_{adom} \varphi[\upsilon]$, iff:
  - *φ is* $R(u)$ and $\upsilon(u) \in I(R)$
  - *φ is* $x=y$ and $\upsilon(x)=\upsilon(y)$
  - *φ is* $\varphi 1 \wedge \varphi 2$ and $I \models_{adom} \varphi 1[\upsilon]$ and $I \models_{adom} \varphi 2[\upsilon]$
  - *φ is* $\varphi 1 \vee \varphi 2$ and $I \models_{adom} \varphi 1[\upsilon]$ or $I \models_{adom} \varphi 2[\upsilon]$

Or: ***υ* designates an answer to *φ* within *I***

Active-domain interpretation (cont'd)

- *I* **satisfies** φ for the valuation υ under the active-domain interpretation, denoted $I \models_{adom} \varphi[\upsilon]$ iff:
  - φ *is* $\neg\ \varphi 1$ and $I \models_{adom} \varphi 1[\upsilon]$ does not hold
  - φ *is* $\forall x\ \varphi 1$ and for any $c \in$ adom(I, q), $I \models_{adom} \varphi 1[\upsilon \cup \{x/c\}]$
  - φ *is* $\exists\ x\ \varphi 1$ and for some $c \in$ adom(I, q), $I \models_{adom} \varphi 1[\upsilon \cup \{x/c\}]$

Or: **υ denotes an answer to φ within *I***

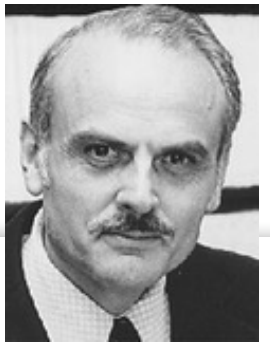
υ extended to also map x into c

- The **answer** to q over I under the adom interpretation is:

$q_{adom}(\mathrm{I}) = \{\upsilon(head(q)) \mid I \models_{adom} \varphi[\upsilon],\ \upsilon$ valuation of $free(\varphi)$ into $adom(q,I)\ \}$

Here, $\upsilon(head(q))$ is an adom(q,I) tuple. Each attribute in $head(q)$ has the value assigned by υ, to the corresponding attribute in a tuple $\upsilon(T)$, where T is a tuple variable in the body of q.

Codd's equivalence theorem



Edgar Codd, Turing Award 1981
Relational data model and relational query languages

Theorem: Relational calculus under the active-domain interpretation is equivalent to relational algebra

- Any relational algebra expression can be equivalently expressed in TRC under the active-domain interpretation
- Any TRC expression under the active-domain interpretation can be equivalently expressed as a relational algebra expression
  - Actually, as **many** relational algebra expressions → optimization of declarative queries!

Relational completeness of a query language: ability to express (relational calculus under adom | relational algebra)

Examples

sid	sname	rating	age
22	dustin	7	45.0
31	lubber	8	55.0
58	rusty	10	55.0
44	ginny	8	36.0

Query 1: $\{P \mid \exists S\,(S \in Sailors \wedge S.\text{rating}=8 \wedge P.\text{name}=S.\text{name})\}$

Query 2: $\{P \mid \exists R\,(R \in Sailors\ \forall\ S \in Sailors,\ R.\text{age}>=S.\text{age} \wedge R.\text{age}=P.\text{age})\}$

More algebra and calculus examples



Exercise *[Ramakrishnan and Gerhke]*

- Suppliers(sid: integer, sname: string, address: string)
- Parts(pid: string, pname: string, color: string)
- Catalog(sid: integer, pid: integer, cost: real)

1. Find the names of suppliers who supply some red part

Relational algebra:

$$\pi_{sname}(\text{Suppliers} \bowtie \text{Catalog} \bowtie \sigma_{color='red'}(\text{Parts}))$$

TRC: $\{ R \mid \exists S \in \text{Suppliers}\ \exists P \in \text{Parts}\ (P.color='red' \wedge \exists C \in \text{Catalog}\ (C.sid=S.sid \wedge C.pid=P.pid \wedge R.name = S.sname)) \}$

Exercise *[Ramakrishnan and Gerhke]*

- Suppliers(sid: integer, sname: string, address: string)
- Parts(pid: string, pname: string, color: string)
- Catalog(sid: integer, pid: integer, cost: real)

2. Find the names of suppliers who supply some red or green part

Relational algebra:

$\pi_{sname}(\text{Suppliers} \bowtie \text{Catalog} \bowtie \sigma_{color='red' \vee color='green'}(\text{Parts}))$

TRC: $\{ R \mid \exists S \in \text{Suppliers}\ \exists P \in \text{Parts}$
$((P.color='red' \vee P.color='green') \wedge$
$\exists C \in \text{Catalog}\ (C.sid=S.sid \wedge C.pid=P.pid \wedge$
$R.name = S.sname)) \}$

Exercise *[Ramakrishnan and Gerhke]*

- Suppliers(sid: integer, sname: string, address: string)
- Parts(pid: string, pname: string, color: string)
- Catalog(sid: integer, pid: integer, cost: real)

3. Find the names of suppliers who supply some red part or are at 221 Packer Avenue

Relational algebra:

$$\pi_{sname}(\text{Suppliers} \bowtie \text{Catalog} \bowtie \sigma_{color='red'}(\text{Parts})) \cup \pi_{sname}(\sigma_{address='221\ Packer\ Avenue'}(\text{Suppliers}))$$

OR

$$\pi_{sname}(\pi_{sid,sname,saddress}(\text{Suppliers} \bowtie \text{Catalog} \bowtie (\sigma_{color='red'}(\text{Parts}))) \cup \sigma_{address='221\ Packer\ Avenue'}(\text{Suppliers}))$$

Exercise *[Ramakrishnan and Gerhke]*

- Suppliers(sid: integer, sname: string, address: string)
- Parts(pid: string, pname: string, color: string)
- Catalog(sid: integer, pid: integer, cost: real)

3. Find the names of suppliers who supply some red part or are at 221 Packer Avenue

TRC: $\{ R \mid \exists S \in \text{Suppliers} \; (\exists P \in \text{Parts} \wedge P.\text{color}=\text{'red'} \wedge \exists C \in \text{Catalog} \; (C.\text{sid}=S.\text{sid} \wedge C.\text{pid}=P.\text{pid})) \vee (S.\text{address}=\text{'221 Packer Avenue'})) \wedge R.\text{name} = S.\text{sname} \}$

Exercise *[Ramakrishnan and Gerhke]*

- Suppliers(sid: integer, sname: string, address: string)
- Parts(pid: string, pname: string, color: string)
- Catalog(sid: integer, pid: integer, cost: real)

4. Find the names of suppliers who supply some red part and some green part

Algebra:

$$\pi_{name}(\pi_{name,id}(\text{Suppliers} \bowtie \text{Catalog} \bowtie \sigma_{color='red'}(\text{Parts})) \cap \pi_{name,id}(\text{Suppliers} \bowtie \text{Catalog} \bowtie \sigma_{color='green'}(\text{Parts}))$$

TRC: $\{ R \mid \exists S \in \text{Suppliers} (\exists P1 \in \text{Parts} \wedge P1.color='red' \wedge \exists C1 \in \text{Catalog} \ (C1.sid=S.sid \wedge C1.pid=P1.pid)) \wedge (\exists P2 \in \text{Parts} \wedge P2.color='green' \wedge \exists C2 \in \text{Catalog} \ (C2.sid=S.sid \wedge C2.pid=P2.pid)) \wedge R.name = S.sname \}$

Exercise *[Ramakrishnan and Gerhke]*

- Suppliers(sid: integer, sname: string, address: string)
- Parts(pid: string, pname: string, color: string)
- Catalog(sid: integer, pid: integer, cost: real)

5. Find the sids of suppliers who supply every part

Algebra:

$$\pi_{sid,pid}(\text{Catalog}) / \pi_{pid}(\text{Parts})$$

TRC: $\{ R \mid \forall P \in \text{Parts } \exists C \in \text{Catalog } (P.pid = C.pid \wedge R.sid = C.sid) \}$

Exercise *[Ramakrishnan and Gerhke]*

- Suppliers(sid: integer, sname: string, address: string)
- Parts(pid: string, pname: string, color: string)
- Catalog(sid: integer, pid: integer, cost: real)

6. Find the sids of suppliers who supply every green part or who supply every red part

Algebra:

$\pi_{sid,pid}(\text{Catalog}) / \pi_{pid}(\sigma_{color='red'}(\text{Parts})) \cup$
$\pi_{sid,pid}(\text{Catalog}) / \pi_{pid}(\sigma_{color='green'}(\text{Parts}))$

TRC: $\{ R \mid (\forall P1 \in \text{Parts}\ (P1.color \neq red \vee \exists C1 \in \text{Catalog}\ (P1.pid=C1.pid \wedge R.sid=C1.sid)\ )\ ) \vee (\forall P2 \in \text{Parts}\ (P2.color \neq green \vee \exists C2 \in \text{Catalog}\ (P2.pid=C2.pid \wedge R.sid=C2.sid)\ )\ )\}$

Exercise *[Ramakrishnan and Gerhke]*

- Suppliers(sid: integer, sname: string, address: string)
- Parts(pid: string, pname: string, color: string)
- Catalog(sid: integer, pid: integer, cost: real)

7. Find pairs of sids such that the first supplier charges more for some part than the second supplier

Algebra:

$\pi_{sid,sid2}$ (Catalog $\bowtie_{pid=pid2 \wedge cost>cost2}$ ρ (Catalog, pid→pid2, cost→cost2))

TRC: { R | ∃C1∈Catalog ∃C2∈Catalog
(C1.pid=C2.pid ∧ C1.cost>C2.cost ∧
R.sid=C1.sid ∧ R.sid2=C2.sid) }

Exercise *[Ramakrishnan and Gerhke]*

- Suppliers(sid: integer, sname: string, address: string)
- Parts(pid: string, pname: string, color: string)
- Catalog(sid: integer, pid: integer, cost: real)

8. Find the pids of parts supplied by at least two different suppliers

Algebra:

π_{pid} (Catalog $\bowtie_{pid=pid2 \wedge sid<>sid2}$ ρ (Catalog, pid→pid2, sid→sid2))

TRC: { R | ∃C1∈Catalog ∃C2∈Catalog
(C1.pid=C2.pid ∧ C1.sid≠C2.sid ∧ R.pid=C1.pid) }

Exercise *[Ramakrishnan and Gerhke]*

- Suppliers(sid: integer, sname: string, address: string)
- Parts(pid: string, pname: string, color: string)
- Catalog(sid: integer, pid: integer, cost: real)

9. Find the pids of the most expensive parts supplied by any supplier called "Yosemite"

Algebra:

$\rho(YParts, \pi_{sid,pid,cost}(\sigma_{sname='Yosemite'}(\text{Suppliers}) \bowtie \text{Catalog}))$

$\rho(CheaperYParts, \pi_{pid}(YParts \bowtie_{sid=sid2,cost<cost2} \rho(YParts, pid \rightarrow pid2, sid \rightarrow sid2, cost \rightarrow cost2)))$

$\pi_{pid}(YParts) \setminus \pi_{pid}(CheaperYParts)$

TRC: $\{ R \mid \exists C1 \in \text{Catalog}\ \exists S \in \text{Suppliers}\ (C1.sid=S.sid \wedge S.sname='Yosemite' \wedge R.pid=C1.pid \wedge \forall C2 \in \text{Catalog}\ (C2.sid \neq S.sid \vee C2.cost \leq C1.cost)) \}$

Questions

